

Model Evaluation

Data Science

Model Evaluation

- classification metrics
- regression metrics
- feature importance
- local explainability
- hypothesis testing (separate slide deck)

Evaluation Scenarios

1. loss function in training: proxy for later test accuracy
2. model validation: decide on acceptance of model changes by means of accuracy measure → improved model vs baseline (current best)
3. test (i.e., out-of-sample) accuracy of predictions: the final goal

Classification Metrics

evaluating model performance for classification tasks

different problems require different metrics

choice of metric depends on:

- class imbalance
- cost of errors
- business or domain goals

example: spam detection vs. disease diagnosis → different priorities

Confusion Matrix

foundation for most classification metrics

for binary classification:

	predicted positive	predicted negative
actual positive	True Positive (TP)	False Negative (FN)
actual negative	False Positive (FP)	True Negative (TN)

other metrics derived from these four values

Accuracy

definition: proportion of correct predictions

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

- simple and intuitive
- but misleading with imbalanced datasets

Precision

definition: of predicted positives, how many are correct?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- use when false positives are costly
- example: email spam filter

Recall (Sensitivity)

definition: of actual positives, how many are detected?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

- use when missing positives is costly
- example: medical diagnosis

Precision vs. Recall Trade-off

- increasing precision often reduces recall
- increasing recall often reduces precision

key idea: threshold tuning controls this trade-off

question to ask: which error matters more?

F1 Score

definition: harmonic mean of precision and recall

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

penalizes extreme imbalance between precision and recall

→ use when balance between precision and recall is needed

Specificity (True Negative Rate)

definition: of actual negatives, how many are correctly identified?

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

- use when false positives are critical
- often paired with recall (sensitivity)

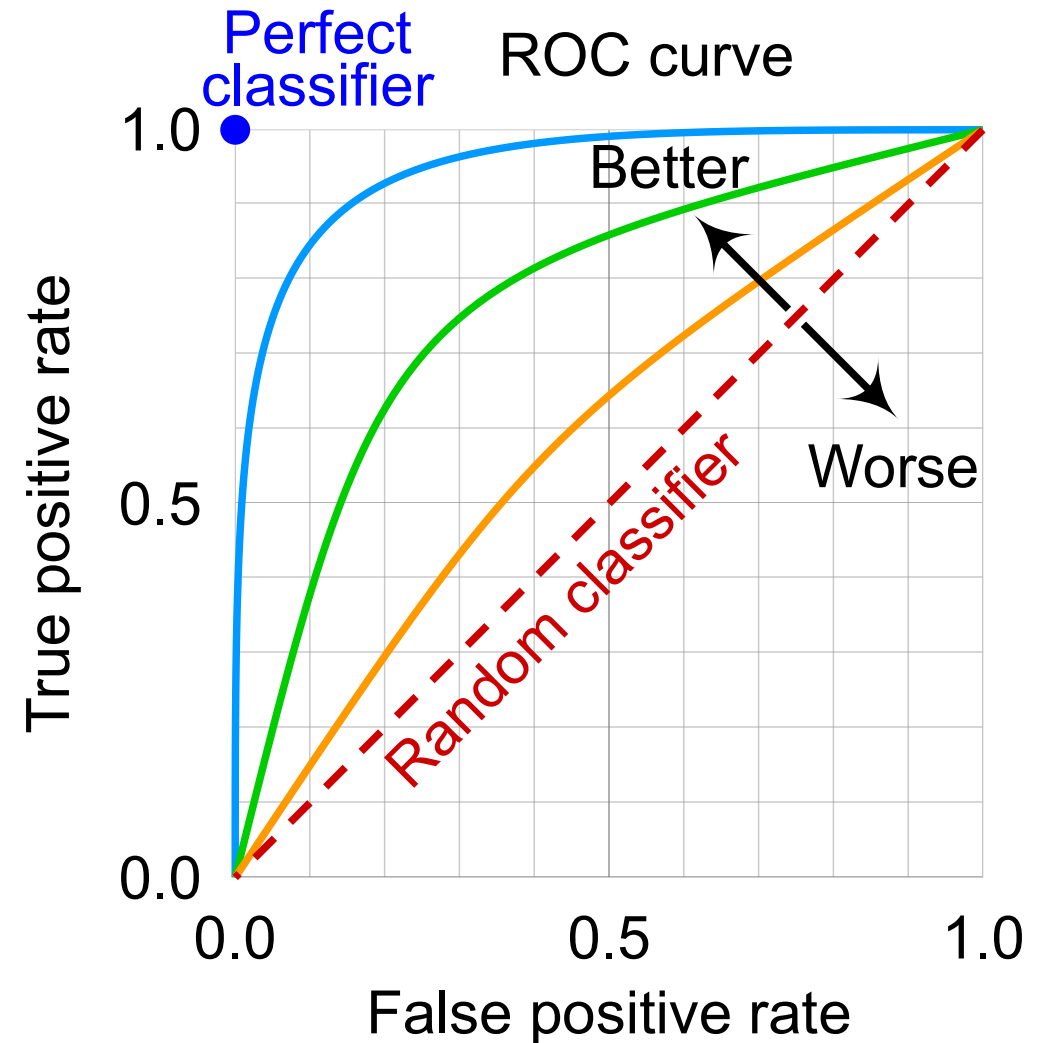
ROC Curve

Receiver Operating Characteristic

plots True Positive Rate (Recall) vs.
False Positive Rate (FPR)

key idea: shows performance across all
thresholds

better model → curve closer to top-left



AUC (Area Under Curve)

definition: area under ROC curve

interpretation:

- 1.0 $\rightarrow$ perfect model
- 0.5 $\rightarrow$ random guessing

advantage: threshold-independent evaluation

Precision-Recall Curve

plots precision vs. recall

important for imbalanced datasets

(ROC can be overly optimistic when classes are skewed)

Choosing the Right Metric

- balanced dataset → accuracy may suffice
- imbalanced dataset → F1, PR curve
- high cost of false positives → precision
- high cost of false negatives → recall

rule of thumb: always align metric with real-world objective

Regression Metrics

evaluating model performance for continuous predictions

errors: deviations between predicted and actual values

→ *How wrong* are the predictions?

choice of metric depends on:

- sensitivity to outliers (some metrics penalize large errors more heavily)
- interpretability
- business or domain goals

Error Basics

residual (error): difference between actual and predicted value

$$\text{residual} = y - \hat{y}$$

- positive $\rightarrow$ underestimation
- negative $\rightarrow$ overestimation

mean residual shows general under- or overestimation:

$$\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y)$$

Mean Squared Error (MSE)

definition: average of squared errors

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- penalizes large errors heavily
- smooth for optimization
- sensitive to outliers

Root Mean Squared Error (RMSE)

definition: square root of MSE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- same units as target variable
- interpretation: typical size of prediction error

Mean Absolute Error (MAE)

definition: average absolute difference between predictions and actual values

aka MAD

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- easy to interpret
- robust to outliers
- does not strongly penalize large errors

MAE vs. MSE/RMSE

MAE: treats all errors equally

MSE/ RMSE: penalize large errors more

rule of thumb:

- use MAE when robustness matters
- use MSE when large errors are critical

R^2 (Coefficient of Determination)

definition: proportion of variance explained by the model

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

interpretation:

- 1.0 $\rightarrow$ perfect fit
- 0.0 $\rightarrow$ same as predicting mean
- < 0 $\rightarrow$ worse than baseline

Mean Absolute Percentage Error (MAPE)

definition: error expressed as percentage

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- easy to interpret (%)
- undefined when $y = 0$
- biased toward small values

Choosing the Right Metric

- MAE: robust, interpretable
- RMSE: penalizes large errors



absolute metrics:
increase with increasing y

- R^2 : explains variance
- MAPE: relative error (%)



relative metrics:
be careful with small y

key question: What kind of errors matter most?

Point Estimates vs. Full PDF

all considered metrics describe point estimates

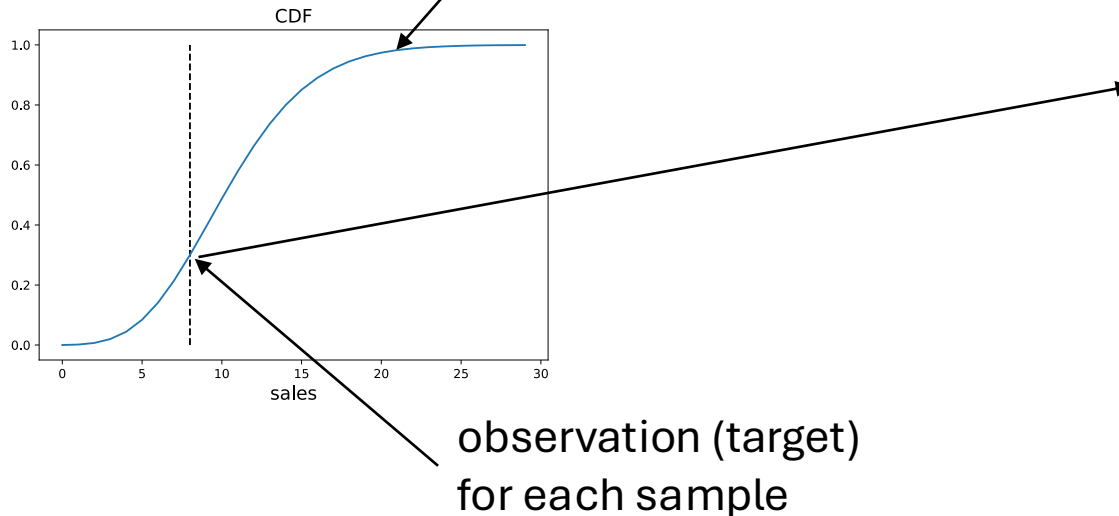
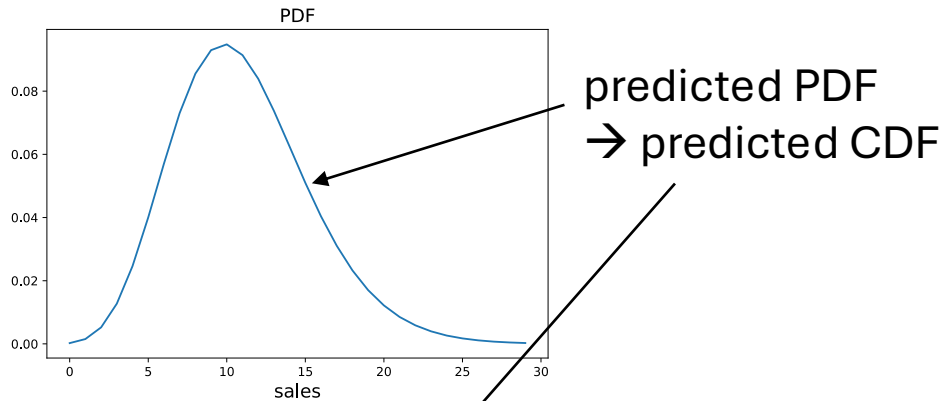
typically for the mean or median of the assumed probability distribution

metrics for evaluation of predictions of full individual (i.e., for each observation) probability distributions [a bit tricky](#)

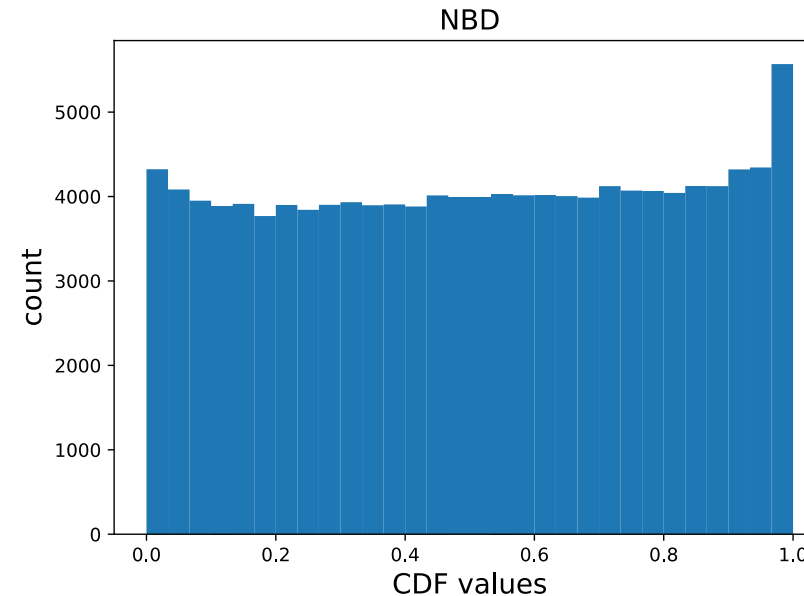
- as discussed before, estimation of full individual PDFs typically requires estimation of additional parameters
- mean estimate, homoscedasticity assumption, and assumption of underlying PDF (for example Gaussian in linear regression) not truly individual

Histogram of CDF Observations

individual prediction/observation:



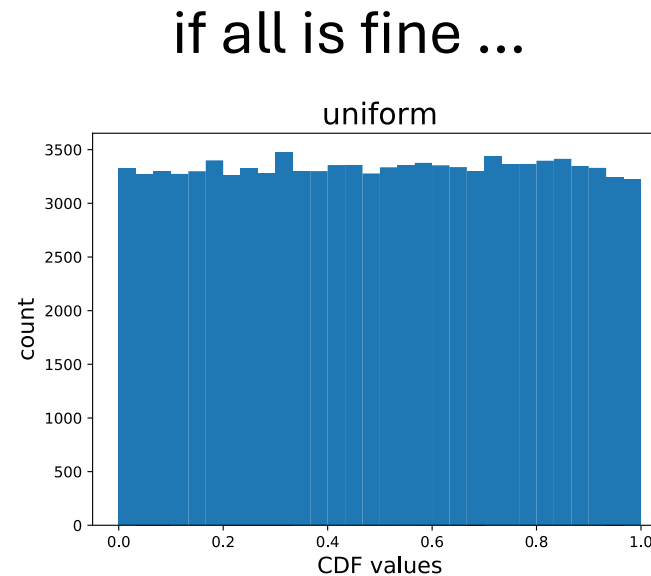
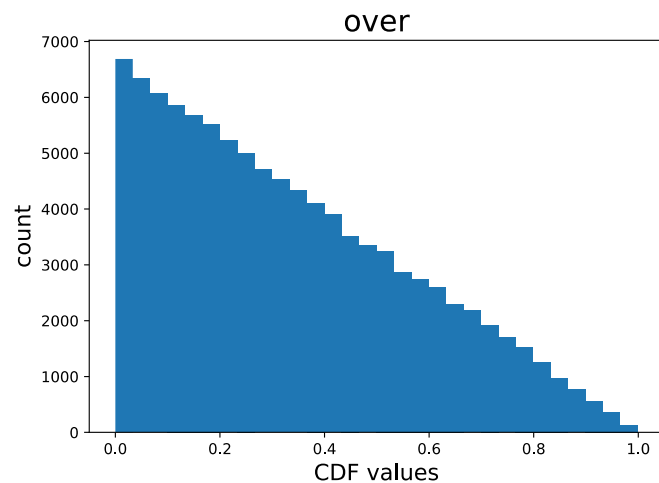
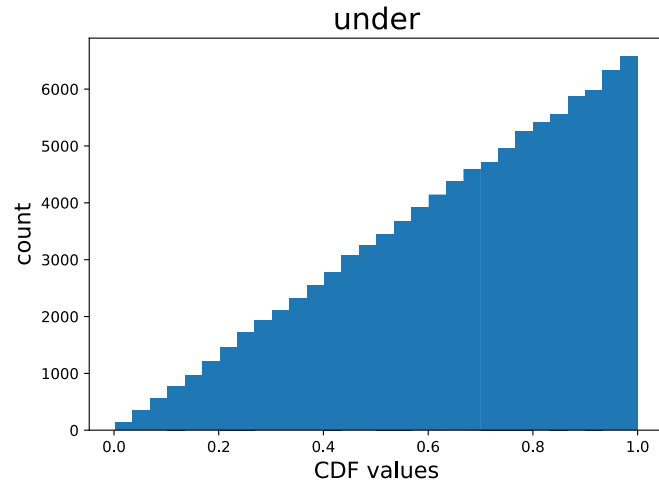
many (or all) predictions/observations:



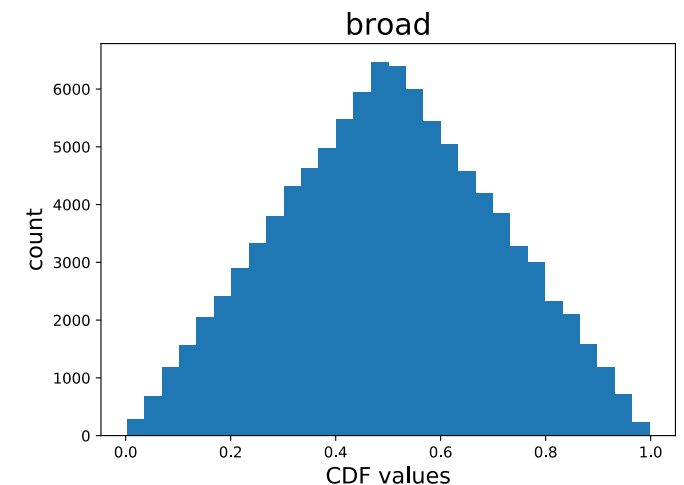
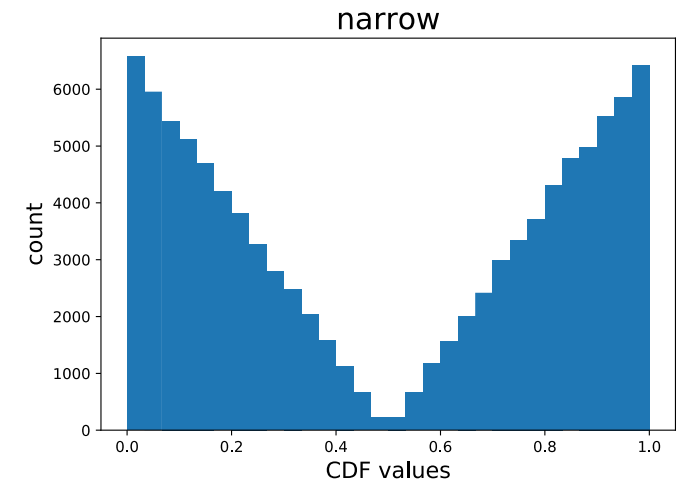
subtlety for discrete PDFs:

histogram filled using random numbers according to the CDF intervals (e.g., for 3 actual sales: random pick between discrete CDF values for 2 and 3)

Interpretation of CDF Histogram



quantitative evaluation: measure deviation from uniform distribution (e.g., earth mover's distance)



Feature Importance

goal: understand which inputs drive model predictions

- improves model interpretability
- helps with feature selection, debugging models, trust & transparency

distinguish:

- model-specific (built-in) vs. model-agnostic (external) methods
- global (overall influence across dataset) vs. local (influence for a single prediction) importance (example: age may be globally important, but not important for a specific individual)

Model-Specific Importance

linear model coefficients

- idea: importance = magnitude of coefficients (larger absolute value → stronger influence)
- important: features must be scaled
- only captures linear relationships

tree-based feature importance (used in decision trees, random forests, gradient boosting)

- idea: importance based on reduction in impurity (e.g., Gini, MSE)
- captures non-linear relationships
- but can favor high-cardinality features

Model-Agnostic Explainability

most prominent method: permutation importance

idea: shuffle one feature and measure performance drop

1. train model
2. shuffle feature values
3. measure performance decrease

larger drop → more important feature

- easy to understand
- but computationally expensive and affected by correlated features

do not confuse explainability with causality
→ explanations can be misleading

Local Explainability

explaining individual predictions:

Why did the model make *this* prediction?

- builds trust in model decisions
- critical in healthcare, finance, legal systems, ...

straight-forward for trees or linear models

but not for complex models like neural networks (black-box models)

→ need for model-agnostic methods to explain black-box models

LIME (Local Interpretable Model-Agnostic Explanations)

idea: approximate model locally with simple model (aka local surrogate)

1. sample nearby data points
2. get predictions
3. fit interpretable model (e.g., linear)

output: feature contributions for one prediction

SHAP (SHapley Additive exPlanations)

idea based on game theory (Shapley values)

key concept: fair contribution of each feature

output:

- contribution values per feature
- sum = model prediction
- positive value → pushes prediction higher
- negative value → pushes prediction lower

consistent and theoretically grounded, works for many model types

LIME vs. SHAP

LIME:

- faster, simpler
- local approximation

SHAP:

- more consistent
- computationally heavier

→ trade-off: speed vs. theoretical guarantees

for both: correlated features complicate interpretation